ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN 2
MÔN NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)

TOPIC 1: Tic-tac-toe (7x7, 9x9, 11x11)

Ngành: KHOA HỌC MÁY TÍNH

GVHD: NGUYỄN QUỐC MINH

—o0o—

Lớp: A01

SV: LÝ QUỐC HÀO (2352294)

SV: NGUYỄN NGỌC PHÁT (2352887)

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

Mục lục

Danh sách sinh viên thực hiện **1**

1 Tổng quan bài tập lớn **2**
1.1 Mô tả về trò chơi Tic-tac-toe . . . 2
1.2 Mô hình trạng thái bàn cờ . . . 2
1.2.1 Biểu diễn trạng thái bàn cờ . . . 2
1.2.2 Cấu trúc cửa sổ trượt tiền tính toán . . . 2
1.2.3 Trạng thái khởi tạo . . . 3
1.2.4 Tập hành động hợp lệ và sinh nước đi ứng viên . . . 3
1.2.5 Hàm chuyển trạng thái (*Transition Model* – `play(move)` và `undo()`) . . . 3
1.2.6 Kiểm tra kết thúc và hàm hữu dụng (*Terminal Test* & *Utility Function*) . . . 3
1.3 Độ phức tạp của bài toán (Level of Challenge) . . . 4
1.3.1 Phân tích thách thức tính toán . . . 4
1.3.2 Các giải pháp kiểm soát độ phức tạp . . . 4

2 Chi tiết các thuật toán **5**
2.1 Kỹ thuật dùng chung: Hàm đánh giá Heuristic và sinh nước đi ứng viên . . . 5
2.1.1 Hàm lượng giá cửa sổ 5 ô . . . 5
2.1.2 Thu hẹp không gian và sắp xếp nước đi . . . 5
2.2 Thuật toán 1: Backtracking . . . 6
2.2.1 Nguyên lý hoạt động . . . 6
2.2.2 Mã giả (Pseudocode) . . . 6
2.3 Thuật toán 2: Alpha-Beta Pruning . . . 6
2.3.1 Nguyên lý hoạt động . . . 6
2.3.2 Mã giả (Pseudocode) . . . 7
2.4 Thuật toán 3: Monte Carlo Tree Search (MCTS) . . . 7
2.4.1 Cấu trúc nút tìm kiếm . . . 7
2.4.2 Bốn giai đoạn của mỗi vòng lặp . . . 8
2.4.3 Quyết định nước đi cuối cùng . . . 8
2.4.4 Mã giả (Pseudocode) . . . 9

3 Thiết kế và Minh họa Giao diện **10**
3.1 Mô hình kiến trúc 3 tầng (3-Layer Architecture) . . . 10
3.1.1 Cơ chế xử lý đa tiến trình không chặn giao diện (Non-blocking Multiprocessing) . . . 10
3.2 Sơ đồ luồng xử lý . . . 11
3.3 Giao diện người dùng . . . 13

4 Thực nghiệm và So sánh các phương pháp **14**
4.1 Môi trường và thiết lập thử nghiệm . . . 15
4.2 So sánh Backtracking và Alpha–Beta ở cùng độ sâu . . . 15
4.3 Đấu đối kháng dưới cùng ngân sách thời gian . . . 16
4.3.1 Thời gian phản hồi và lượng tìm kiếm . . . 16
4.3.2 Kết quả thắng, hòa, thua . . . 17
4.3.3 Nhật ký 36 ván đấu . . . 17
4.4 Mức tiêu thụ bộ nhớ . . . 18
4.5 Đánh giá tổng hợp và giới hạn . . . 18

5 Tổng kết **19**
5.1 Kết luận và hạn chế . . . 19
5.2 Hướng phát triển . . . 19
5.3 Hướng dẫn cài đặt và khởi chạy . . . 19

6 TÀI LIỆU THAM KHẢO **20**

Danh sách sinh viên thực hiện

STT	Họ và tên	MSSV	Mức độ hoàn thành
1	Lý Quốc Hào	2352294	100%
2	Nguyễn Ngọc Phát	2352887	100%

1 Tổng quan bài tập lớn

1.1 Mô tả về trò chơi Tic-tac-toe

Trò chơi Tic-tac-toe truyền thống được chơi trên bàn cờ 3×3, trong đó người chơi thắng khi tạo được ba quân liên tiếp ($k = 3$). Tổng số cấu hình có thể biểu diễn bằng cách gán ba giá trị cho 9 ô là $3^9 = 19{,}683$. Đây là cận trên thô, bao gồm cả những cấu hình không thể đạt được theo luật chơi. Nếu cả hai bên chơi tối ưu, ván đấu kết thúc hòa.

Trong đề tài này, bài toán được mở rộng thành họ trò chơi $(N, N, 5)$ trên các bàn cờ kích thước $N \times N$, với $N \in \{7, 9, 11\}$ (`SIZES = (7, 9, 11)`). Trò chơi có các đặc điểm và quy tắc sau:

- **Luật chơi cơ bản:** Đây là trò chơi đối kháng giữa hai người, có tổng bằng không, thông tin hoàn hảo và tất định. Quân X đi trước, quân O đi sau; hai bên luân phiên đặt quân vào các ô trống.
- **Điều kiện thắng (`WIN_LENGTH = 5`):** Trên các bàn cờ lớn, điều kiện thắng được chuẩn hóa là $k = 5$. Người chơi thắng khi tạo được ít nhất năm quân liên tiếp cùng màu theo một trong bốn hướng: ngang $(0, 1)$, dọc $(1, 0)$, chéo chính $(1, 1)$ hoặc chéo phụ $(1, -1)$.
- **Luật Freestyle (không cấm):** Chuỗi từ sáu quân liên tiếp trở lên (*overline*) vẫn được tính là thắng; chuỗi năm quân bị chặn ở cả hai đầu cũng được tính là thắng. Trò chơi không áp dụng các luật cấm nước đôi như *double-three* hoặc *double-four*.
- **Điều kiện hòa:** Ván đấu kết thúc hòa khi toàn bộ N^2 ô đã được lấp đầy (`move_count == size * size`) mà chưa có người chơi nào tạo được chuỗi năm quân liên tiếp.

1.2 Mô hình trạng thái bàn cờ

Không gian trạng thái của trò chơi được biểu diễn bởi lớp `Board`. Mô hình này lưu trạng thái bàn cờ, lượt đi, các bộ đếm cửa sổ và điểm lượng giá; đồng thời hỗ trợ sinh nước đi, chuyển trạng thái và hoàn tác nước đi để phục vụ thuật toán tìm kiếm.

1.2.1 Biểu diễn trạng thái bàn cờ

- **Mảng một chiều:** Bàn cờ kích thước $N \times N$, với $N \in \{7, 9, 11\}$, được trải phẳng thành mảng một chiều `cells` có độ dài N^2.
- **Ánh xạ tọa độ:** Ô ở hàng r, cột c (với $0 \le r, c < N$) được ánh xạ tới chỉ số tuyến tính:

$$\text{index} = r \times N + c.$$

- **Giá trị ô:** Mỗi phần tử `cells[index]` nhận một trong ba giá trị thuộc tập $\{0, 1, -1\}$: ô trống, quân X hoặc quân O.

1.2.2 Cấu trúc cửa sổ trượt tiền tính toán

Tiền tính toán và lưu đệm (`@lru_cache`): Các cửa sổ gồm năm ô liên tiếp hợp lệ (*windows*) và danh sách cửa sổ đi qua từng ô (`memberships[cell]`) được tính một lần cho mỗi kích thước bàn cờ và lưu vào bộ nhớ đệm.

Trên bàn cờ $N \times N$, số cửa sổ theo bốn hướng ngang, dọc, chéo chính và chéo phụ là:

$$W(N) = 2N(N-4) + 2(N-4)^2.$$

Do đó, bàn cờ 7×7 có 60 cửa sổ, bàn cờ 9×9 có 140 cửa sổ và bàn cờ 11×11 có 252 cửa sổ.

Cập nhật điểm tăng dần (*Incremental Evaluation*): Đối tượng `Board` lưu số quân X (`x_counts`) và quân O (`o_counts`) trong mỗi cửa sổ, cùng tổng điểm `score`. Khi một quân được đặt hoặc gỡ, chỉ các cửa sổ chứa ô đó cần cập nhật. Vì mỗi ô thuộc tối đa 20 cửa sổ, việc cập nhật điểm mất $O(1)$ thời gian với độ dài chuỗi thắng cố định là 5, thay vì phải quét lại toàn bộ bàn cờ trong $O(N^2)$ thời gian.

1.2.3 Trạng thái khởi tạo

Trạng thái đầu ván được thiết lập như sau:

- `cells = [0] * (N * N)`: toàn bộ ô trên bàn cờ đều trống.
- `to_move = +1`: quân X đi trước.
- `move_count = 0`: chưa có nước đi nào được thực hiện.
- `score = 0`: điểm lượng giá ban đầu cân bằng.
- `winner = 0`: chưa có người chiến thắng.

1.2.4 Tập hành động hợp lệ và sinh nước đi ứng viên

Tập nước đi hợp lệ đầy đủ gồm mọi chỉ số i thỏa mãn `cells[i] == 0`. Để giảm hệ số phân nhánh khi tìm kiếm trên bàn cờ có 49, 81 hoặc 121 ô, mô-đun `game/candidates.py` thu hẹp tập nước đi ứng viên qua các bước sau:

1. **Ưu tiên nước đi bắt buộc (*Forced Moves*):** Nếu bên đang tới lượt có nước thắng trực tiếp bằng cách tạo đủ năm quân liên tiếp, chỉ trả về các nước thắng đó. Nếu không có nước thắng trực tiếp nhưng đối phương có thể thắng ngay ở lượt kế tiếp, chỉ trả về các ô cần đánh để chặn. Các nước thắng hoặc chặn bắt buộc không bị giới hạn bởi `max_candidates`.
2. **Nước đi khai cuộc (*Opening*):** Nếu bàn cờ còn trống (`move_count == 0`), chọn ngay ô chính giữa bàn cờ, có chỉ số tuyến tính `(size * size) // 2`.
3. **Lọc theo khoảng cách Chebyshev ($R = 2$):** Chỉ giữ các ô trống cách không quá hai ô theo cả hàng và cột so với ít nhất một quân đã đánh. Với hai ô (r_1, c_1) và (r_2, c_2), điều kiện lọc là:
$$\max\big(|r_1 - r_2|, |c_1 - c_2|\big) \leq 2.$$
4. **Xếp hạng và giới hạn Top-K:** Chấm điểm cục bộ các ô ứng viên theo `priority`, sắp xếp giảm dần và giữ tối đa K nước có điểm cao nhất. Giá trị `max_candidates` là 12 trên GUI và 8 trong benchmark nhanh.

1.2.5 Hàm chuyển trạng thái (*Transition Model* – `play(move)` và `undo()`)

Thực hiện nước đi:

1. Đặt quân hiện tại vào ô được chọn: `cells[move] = to_move`.
2. Lưu thông tin cần thiết để hoàn tác (gồm `move`, `last_move` và `winner`) vào ngăn xếp `history`, rồi tăng `move_count` thêm một.
3. Duyệt các cửa sổ chứa ô vừa đánh trong `memberships[move]` (tối đa 20 cửa sổ), cập nhật `x_counts` hoặc `o_counts` và điều chỉnh `score` theo chênh lệch điểm của các cửa sổ đó.
4. Đổi lượt người chơi: `to_move = -to_move`.

Hoàn tác nước đi : Lấy trạng thái gần nhất từ `history`, xóa quân vừa đi khỏi `cells`, rồi khôi phục các bộ đếm cửa sổ, `score`, `winner` và `to_move` về trạng thái trước nước đi.

Cơ chế `play/undo` cập nhật trạng thái tại chỗ, nhờ đó các thuật toán duyệt cây như Backtracking và Alpha-Beta có thể xét các nút con mà không cần tạo bản sao sâu của toàn bộ bàn cờ.

1.2.6 Kiểm tra kết thúc và hàm hữu dụng (*Terminal Test* & *Utility Function*)

Kiểm tra kết thúc (*Terminal Test*):

- **Thắng:** Sau khi thực hiện `play(move)`, nếu một cửa sổ $w \in$ `memberships[move]` có đủ năm quân cùng màu (bộ đếm tương ứng bằng 5), ván đấu kết thúc và `winner` được gán bằng người vừa đi.
- **Hòa** (`is_draw`)**:** Ván đấu hòa khi `move_count == size * size` và `winner == 0`.

Hàm hữu dụng (*Utility Function* – `ai/minimax.py`): Tại nút lá kết thúc trong tìm kiếm Negamax:

- Nếu `board.winner != 0`, trả về điểm thắng/thua có độ lớn `WIN_SCORE - ply`, với `WIN_SCORE = 1,000,000`. Theo quy ước dấu của Negamax, giá trị được biểu diễn tương ứng là $\pm(1{,}000{,}000 - \text{ply})$. Thành phần `ply` giúp AI ưu tiên đường thắng ngắn hơn và kéo dài đường đi khi không tránh khỏi thất bại.
- Nếu `board.is_draw`, trả về giá trị 0.

1.3 Độ phức tạp của bài toán (Level of Challenge)

Kích thước bàn cờ	Số ô N^2	Hệ số nhánh đầu trận b_0	Cận trên thô số cấu hình 3^{N^2}	Số thứ tự điền đủ bàn cờ $(N^2)!$
3×3	9	9	$3^9 \approx 1.97\times10^4$	$9! \approx 3.63\times10^5$
7×7	49	49	$3^{49} \approx 2.39\times10^{23}$	$49! \approx 6.08\times10^{62}$
9×9	81	81	$3^{81} \approx 4.43\times10^{38}$	$81! \approx 5.80\times10^{120}$
11×11	121	121	$3^{121} \approx 5.39\times10^{57}$	$121! \approx 8.09\times10^{200}$

Bảng 1.1: *Độ phức tạp tính toán của Tic-tac-toe mở rộng trên các cấu hình bàn cờ.*

Lưu ý: 3^{N^2} là cận trên thô đếm mọi cách gán ba giá trị cho các ô, kể cả cấu hình không thể đạt được theo luật luân phiên. $(N^2)!$ đếm số thứ tự có thể lấp đầy toàn bộ bàn cờ nếu chưa dừng sớm khi có người thắng; đây là thước đo quy mô tổ hợp, không phải số nút thực tế của một ván đấu.

1.3.1 Phân tích thách thức tính toán

Trên bàn cờ Tic-tac-toe truyền thống 3×3, cận trên số thứ tự điền đủ bàn cờ chỉ là $9! \approx 3.63\times10^5$. Vì vậy, thuật toán Backtracking hoặc Minimax cơ bản có thể duyệt vét cạn không gian tìm kiếm nhỏ này; thời gian thực tế phụ thuộc vào cách cài đặt và phần cứng.

Khi mở rộng lên bàn 7×7, số thứ tự điền đủ bàn cờ đã vào khoảng 6.08×10^{62}; với bàn 11×11, con số này tăng lên khoảng 8.09×10^{200}. Để so sánh về bậc độ lớn, ước lượng thường dùng cho cây trò chơi Cờ vua vào khoảng 10^{123}, còn số nguyên tử trong vũ trụ quan sát được vào khoảng 10^{80}. Ngay cả khi chỉ tìm kiếm thuần túy đến độ sâu $d = 6$ trên bàn 11×11, số chuỗi nước đi tiềm năng đã là:

$$121\times120\times119\times118\times117\times116 = 2{,}767{,}192{,}908{,}480 \approx 2.77\times10^{12}.$$

Quy mô này vượt quá khả năng đáp ứng thời gian thực nếu không hạn chế tìm kiếm.

1.3.2 Các giải pháp kiểm soát độ phức tạp

Do không thể duyệt vét cạn cây trò chơi trên các bàn cờ lớn, tác tử AI kết hợp các cơ chế giảm tải sau:

1. **Giới hạn độ sâu và tìm kiếm sâu dần (`max_depth` và *Iterative Deepening*):** Thay vì tìm tới trạng thái kết thúc, thuật toán giới hạn độ sâu, đánh giá các nút lá bằng hàm heuristic dựa trên điểm thế cờ đã được duy trì. Tìm kiếm sâu dần hoàn tất từng độ sâu trước khi tăng độ sâu tiếp theo, nhờ đó có thể trả về kết quả tốt nhất đã tìm được trong giới hạn thời gian (`time_limit`).

2. **Ưu tiên nước đi bắt buộc:** Nếu bên đang tới lượt có nước thắng trực tiếp, thuật toán chỉ giữ các nước thắng đó. Nếu không, nhưng đối phương có thể thắng ngay ở lượt kế tiếp, thuật toán chỉ giữ các nước cần thiết để chặn. Các nước thắng hoặc chặn bắt buộc không bị giới hạn bởi `max_candidates`.

3. **Xử lý nước đi khai cuộc:** Nếu bàn cờ còn trống (`move_count == 0`), thuật toán chọn ngay ô chính giữa, có chỉ số tuyến tính `(size * size) // 2`.

4. **Lọc theo khoảng cách Chebyshev ($R = 2$):** Chỉ giữ ô trống cách không quá hai ô theo cả hàng và cột so với ít nhất một quân đã đánh. Với hai ô (r_1, c_1) và (r_2, c_2), điều kiện lọc là:

$$\max\big(|r_1 - r_2|, |c_1 - c_2|\big) \le 2.$$

5. **Xếp hạng và giới hạn Top-K:** Các ứng viên được sắp xếp giảm dần theo điểm cục bộ `priority` và giữ tối đa K nước có điểm cao nhất. Trong cấu hình nêu ở mục 1.2, $K = 12$ trên GUI và $K = 8$ trong benchmark nhanh. Khi đó, đối với các nút không thuộc nhánh xử lý nước khẩn cấp, hệ số nhánh hiệu dụng được giới hạn bởi $b_{\text{eff}} \le K$. Với cây tìm kiếm bị giới hạn ứng viên, số nút ở độ sâu d có cận trên cấp độ $O(K^d)$. Cận này chỉ áp dụng cho phần cây có nhánh bị giới hạn bởi K; các nút xử lý nước khẩn cấp có thể sinh hơn K nước đi nên không thuộc cận này. Đây không phải là cận cho toàn bộ không gian trạng thái của trò chơi.

2 Chi tiết các thuật toán

2.1 Kỹ thuật dùng chung: Hàm đánh giá Heuristic và sinh nước đi ứng viên

Trước khi trình bày ba thuật toán tìm kiếm, hệ thống sử dụng hai mô-đun cốt lõi dùng chung: lượng giá trạng thái trong `ai/evaluation.py` và tối ưu hóa không gian duyệt trong `game/candidates.py`. Đây là các thiết kế của nhóm nhằm giảm ảnh hưởng của bùng nổ tổ hợp trên các bàn cờ mở rộng.

2.1.1 Hàm lượng giá cửa sổ 5 ô

Thay vì quét lại toàn bộ bàn cờ để đếm các chuỗi quân sau mỗi nước đi, hệ thống biểu diễn bàn cờ bằng tập hợp các cửa sổ hợp lệ gồm năm ô liên tiếp theo chiều ngang, dọc hoặc chéo. Các bộ đếm của cửa sổ được cập nhật tăng dần khi một quân được đánh.

Xét một cửa sổ w gồm năm ô liên tiếp. Gọi x_w và o_w lần lượt là số quân X và O trong cửa sổ đó. Nếu cửa sổ chứa quân của cả hai bên ($x_w > 0 \wedge o_w > 0$), nó không còn khả năng tạo thành năm quân liên tiếp cho bên nào, nên điểm cửa sổ bằng 0. Ngược lại, điểm được tính từ mảng trọng số:

$$\texttt{WEIGHTS} = (0, 2, 16, 128, 4096, 1_000_000).$$

Đặt $W_i = \texttt{WEIGHTS}[i]$, điểm cửa sổ là:

$$\text{score}(w) = \begin{cases} 0, & x_w > 0 \wedge o_w > 0, \\ W_{x_w} - W_{o_w}, & \text{otherwise.} \end{cases}$$

Tổng điểm bàn cờ được duy trì bằng tổng điểm của các cửa sổ:

$$\texttt{board.score} = \sum_{w} \text{score}(w).$$

Hàm lượng giá chuẩn hóa điểm theo góc nhìn của người sắp đi, với player $\in \{+1, -1\}$:

$$\text{evaluate}(\text{board}, \text{player}) = \text{clamp}\big(\texttt{board.score} \times \text{player}, -500{,}000, 500{,}000\big).$$

Giới hạn này giữ trị tuyệt đối của điểm heuristic thấp hơn điểm dành cho trạng thái thắng tuyệt đối (1,000,000).

Ý nghĩa thiết kế: Các cửa sổ năm ô chồng lấn lên nhau. Vì vậy, một chuỗi có tiềm năng như ba hoặc bốn quân chưa bị chặn có thể đóng góp điểm qua nhiều cửa sổ liên tiếp, giúp phân biệt thế cờ mạnh với thế cờ bị chặn mà không cần nhiều nhánh điều kiện riêng. Cách tính theo cửa sổ cũng ghi nhận được các mẫu có khoảng trống bên trong, chẳng hạn `X X . X`.

2.1.2 Thu hẹp không gian và sắp xếp nước đi

Để giảm hệ số phân nhánh và đưa các nước đi có triển vọng lên trước, hàm sinh ứng viên (`generate` trong `game/candidates.py`) áp dụng các bước sau:

1. **Lọc nước đi ép buộc:** Tìm các cửa sổ có bốn quân cùng phe và không có quân đối phương để xác định nước thắng trực tiếp (`winning_moves`). Nếu người đang tới lượt có nước thắng ngay, chỉ trả về các nước thắng đó. Nếu không, nhưng đối phương có thể thắng ngay ở lượt kế tiếp, chỉ trả về các ô cần chặn. Các nước thắng hoặc chặn bắt buộc này không bị giới hạn bởi `max_candidates`.

2. **Nước đi khai cuộc:** Nếu bàn cờ còn trống (`move_count == 0`), chọn ô chính giữa bằng chỉ số tuyến tính `(size * size) // 2`.

3. **Lọc lân cận Chebyshev:** Chỉ giữ các ô trống cách không quá hai ô theo khoảng cách Chebyshev so với ít nhất một quân đã đánh:
$$\max\big(|\Delta r|, |\Delta c|\big) \le 2.$$

4. **Chấm điểm và giới hạn ứng viên (*Move Ordering & Candidate Cap*):** Với mỗi ô ứng viên m, gọi a_w và b_w lần lượt là số quân của phe đang xét và phe đối phương trong cửa sổ w chứa m. Đặt $W_i = \texttt{WEIGHTS}[i]$; điểm ưu tiên chiến thuật được tính như sau:
$$\text{priority}(m) = \sum_{w \ni m} \left[\mathbb{I}(b_w = 0)\, 2\big(W_{a_w+1} - W_{a_w}\big) + \mathbb{I}(a_w = 0)\,\big(W_{b_w+1} - W_{b_w}\big)\right].$$
Số hạng thứ nhất đánh giá tiềm năng tấn công và được nhân đôi; số hạng thứ hai đánh giá tiềm năng phòng thủ. Khi ưu tiên bằng nhau, các ô gần tâm bàn cờ được xếp trước. Sau đó, danh sách được sắp xếp giảm dần theo điểm ưu tiên và cắt còn tối đa `max_candidates`: $K = 12$ trên GUI hoặc $K = 8$ trong benchmark nhanh.

2.2 Thuật toán 1: Backtracking

File cài đặt: `ai/minimax.py` và `ai/backtracking.py` (cấu hình `pruning = False`).

2.2.1 Nguyên lý hoạt động

- **Mô hình Negamax:** Đây là biến thể của Minimax dựa trên tính chất đối xứng của trò chơi tổng bằng không (*zero-sum*):
$$\max(a,b) = -\min(-a,-b).$$
Nhờ đó, một hàm đệ quy duy nhất có thể dùng chung cho cả hai người chơi X ($+1$) và O (-1).

- **Quay lui tại chỗ (*In-place Backtracking*):** Thuật toán duyệt cây trò chơi theo chiều sâu (DFS) trực tiếp trên một đối tượng `Board` duy nhất. Với mỗi nước đi, thuật toán thực hiện `board.play(move)`, gọi đệ quy với giá trị đã đổi dấu $-$`search(depth - 1, ...)`, rồi hoàn tác bằng `board.undo()` trong khối `try...finally`. Đối với các nút thông thường, mỗi mức đệ quy lưu tối đa K ứng viên nên bộ nhớ phụ trợ có độ phức tạp $O(dK)$, tương đương $O(d)$ khi K là hằng số. Tại các nút xử lý nước thắng hoặc chặn bắt buộc, số ứng viên có thể vượt K; nếu mức i lưu K_i ứng viên thì cận tổng quát là $O(\sum_{i=1}^{d} K_i)$ (tối đa $O(dK_{\max})$, với $K_{\max}$ là số ứng viên lớn nhất ở một mức). Cách làm này cũng tránh chi phí sao chép sâu ở mỗi nút.

- **Tìm kiếm sâu dần (*Iterative Deepening*):** Khi có giới hạn thời gian (`time_limit`), thuật toán tăng dần độ sâu từ 1 đến `max_depth`. Nếu hết thời gian và ngoại lệ `SearchInterrupted` xảy ra giữa chừng, hệ thống trả về nước đi tốt nhất của độ sâu hoàn tất gần nhất (`completed_depth`). Nếu tìm thấy nhánh thắng tuyệt đối, vòng lặp dừng sớm khi:
$$|\texttt{best_value}| \geq \texttt{WIN_SCORE} - N^2.$$

2.2.2 Mã giả (Pseudocode)

```
def BACKTRACKING_NEGAMAX(board, depth, ply):
    check_timeout_or_cancel()
    stats.nodes += 1

    if board.winner != EMPTY:
        return -WIN_SCORE + ply
    if board.is_draw:
        return 0
    if depth == 0:
        return evaluate(board, board.to_move)

    best = float("-inf")
    for move in generate_candidates(board):
        board.play(move)
        try:
            value = -BACKTRACKING_NEGAMAX(board, depth - 1, ply + 1)
        finally:
            board.undo()
        best = max(best, value)

    return best
```

2.3 Thuật toán 2: Alpha-Beta Pruning

File cài đặt: `ai/minimax.py` và `ai/alphabeta.py` (cấu hình `pruning = True`).

2.3.1 Nguyên lý hoạt động

Alpha-Beta kế thừa lõi Negamax, hàm `evaluate`, bộ sinh nước ứng viên `generate` và cơ chế tìm kiếm sâu dần (*Iterative Deepening*) của Backtracking. Đồng thời, thuật toán duy trì cửa sổ hai cận $[\alpha, \beta]$:

- α là cận dưới, biểu thị điểm tốt nhất mà người đang đi tại nút hiện tại đã bảo đảm đạt được qua các nước đã xét.

- β là cận trên, biểu thị ngưỡng điểm cao nhất mà đối thủ ở nút cha còn cho phép người đang đi đạt tới trước khi chọn một phương án khác.

Theo chuẩn Negamax, khi truyền cửa sổ xuống nút con, hai cận được đảo dấu và đổi vị trí:

$$[\alpha, \beta] \longrightarrow [-\beta, -\alpha].$$

Sau mỗi nhánh con, thuật toán cập nhật α theo điểm tốt nhất hiện có:

$$\text{best} \leftarrow \max(\text{best}, \text{value}), \qquad \alpha \leftarrow \max(\alpha, \text{best}).$$

Điều kiện cắt tỉa (*Beta Cutoff*): Nếu $\alpha \geq \beta$, nhánh hiện tại đã đạt ngưỡng mà đối thủ ở nút cha có thể tránh bằng lựa chọn khác. Các nước còn lại tại nút này không thể làm thay đổi kết quả minimax, nên thuật toán tăng `stats.cutoffs` thêm 1 và thoát vòng lặp bằng `break`.

Sắp xếp nước đi (*Move Ordering*): Hàm `priority` ở mục 2.1 đưa các nước tấn công và phòng thủ triển vọng lên đầu danh sách ứng viên. Khi nước đi tốt được xét sớm, α thường tăng nhanh; nhờ đó điều kiện $\alpha \geq \beta$ có thể được kích hoạt sớm và nhiều nhánh còn lại được cắt bỏ. Hiệu quả cắt tỉa phụ thuộc vào chất lượng thứ tự nước đi.

Tính tương đương toán học: Khi chạy ở cùng độ sâu cố định (`iterative = False`, `time_limit = 0`) và dùng cùng tập ứng viên, hàm lượng giá, điều kiện kết thúc, Alpha-Beta trả về chính xác cùng điểm minimax với Backtracking. Alpha–Beta trả về kết quả Minimax chính xác trong cây tìm kiếm được giới hạn bởi độ sâu và tập ứng viên đã chọn. Vì vậy, tính tối ưu chỉ được bảo đảm trong phạm vi cây tìm kiếm này. Nếu có nhiều nước đồng điểm, lựa chọn cụ thể có thể khác do thứ tự duyệt. Số nút (`nodes`) thường giảm đáng kể khi nước đi được sắp xếp tốt, nhưng mức giảm phụ thuộc vào thế cờ và không được bảo đảm trong mọi trường hợp.

2.3.2 Mã giả (Pseudocode)

```
def ALPHABETA_NEGAMAX(board, depth, alpha, beta, ply):
    check_timeout_or_cancel()
    stats.nodes += 1

    if board.winner != EMPTY:
        return -WIN_SCORE + ply
    if board.is_draw:
        return 0
    if depth == 0:
        return evaluate(board, board.to_move)

    best = float("-inf")
    for move in generate_candidates(board):
        board.play(move)
        try:
            value = -ALPHABETA_NEGAMAX(
                board, depth - 1, -beta, -alpha, ply + 1
            )
        finally:
            board.undo()

        best = max(best, value)
        alpha = max(alpha, best)
        if alpha >= beta:
            stats.cutoffs += 1
            break

    return best
```

2.4 Thuật toán 3: Monte Carlo Tree Search (MCTS)

File cài đặt: `ai/mcts.py` và `ai/evaluation.py`.

2.4.1 Cấu trúc nút tìm kiếm

Mỗi nút `Node` lưu trạng thái cần thiết cho quá trình tìm kiếm:

- `player_just_moved`: người chơi vừa thực hiện nước đi dẫn đến nút này.
- `move`: tọa độ nước đi tạo ra nút; nút gốc không có nước đi.
- `visits` ($N(v)$): số lần nút được thăm.
- `total` ($W(v)$): tổng phần thưởng tích lũy, tính theo góc nhìn của `player_just_moved`.
- `untried`: danh sách nước đi ứng viên tại nút chưa được mở rộng.
- `children`: danh sách các nút con đã được mở rộng và ghi nhận kết quả mô phỏng.

2.4.2 Bốn giai đoạn của mỗi vòng lặp

Mỗi vòng lặp (*iteration*) thực hiện lần lượt bốn giai đoạn sau:

1. **Selection (Chọn nút):** Bắt đầu từ nút gốc. Khi nút hiện tại chưa kết thúc, đã dùng hết các nước trong `untried` và còn nút con, chọn nút con v_i có giá trị UCB1/UCT cao nhất:

$$\mathrm{UCT}(v_i) = \frac{W(v_i)}{N(v_i)} + C\sqrt{\frac{\ln N(v)}{N(v_i)}}, \qquad C = \sqrt{2} \approx 1.41421356,$$

   trong đó v là nút cha. Vì $W(v_i)$ được tính theo góc nhìn của `player_just_moved` tại nút con, cũng chính là người đang chọn nước tại nút cha, số hạng khai thác có thể dùng trực tiếp mà không cần đảo dấu.

2. **Expansion (Mở rộng):** Nếu nút hiện tại chưa kết thúc và `untried` còn nước đi, chọn ngẫu nhiên một nước $m =$ `rng.choice(node.untried)`, thực hiện `board.play(m)` và tạo nút con. Nút con tạm thời được đưa vào đường đi `path`, nhưng chỉ được thêm vào `node.children` sau khi rollout hoàn tất; nước đi cũng chỉ khi đó mới bị xóa khỏi `untried`. Nhờ vậy, nếu mô phỏng bị ngắt giữa chừng thì cây không lưu lại nút con chưa được thăm ($N(v_i) = 0$), vốn có thể gây chia cho 0 khi tính UCT.

3. **Simulation / Rollout (Mô phỏng có định hướng):** Từ nút vừa mở rộng, mô phỏng ván đấu bằng cách chọn ngẫu nhiên trong các nước do `generate` sinh ra. Tập ứng viên áp dụng cơ chế phát hiện nước thắng/chặn thắng bắt buộc, lọc lân cận Chebyshev với bán kính $R = 2$ và giới hạn tối đa K nước đối với các trường hợp thông thường. Các nước thắng hoặc chặn bắt buộc không bị giới hạn bởi `max_candidates`. Mô phỏng dừng khi ván đấu kết thúc hoặc đạt `rollout_limit = 60` bước.

   Nếu chạm giới hạn 60 bước mà ván chưa kết thúc (*heuristic rollout cutoff*), hàm `rollout_value` ánh xạ điểm heuristic về khoảng $[0,1]$ thay vì mặc định coi là hòa:

$$V(\text{board}, p) = \begin{cases} 1.0, & \text{nếu } \texttt{winner} = p, \\ 0.0, & \text{nếu } \texttt{winner} = -p, \\ 0.5, & \text{nếu ván hòa}, \\ 0.5 + 0.5\tanh\left(\dfrac{\text{evaluate}(\text{board}, p)}{8192.0}\right), & \text{nếu rollout bị cắt ngắn khi ván chưa kết thúc}, \end{cases}$$

   trong đó p là người chơi dùng để tính phần thưởng. Hàm này trả về giá trị lớn hơn 0.5 khi thế cờ có lợi cho p và nhỏ hơn 0.5 khi bất lợi.

4. **Backpropagation (Lan truyền ngược):** Duyệt ngược các nút trong `path`. Với mỗi nút `visited`, tăng `visited.visits` lên 1 và cộng vào `visited.total` phần thưởng theo người chơi vừa đi:

   `visited.total` ← `visited.total` + `rollout_value(board, visited.player_just_moved)`.

   Sau khi cập nhật thống kê, gọi `board.undo()` đúng bằng tổng số nước đã thực hiện trong Selection, Expansion và Rollout để khôi phục bàn cờ gốc.

2.4.3 Quyết định nước đi cuối cùng

Sau khi hoàn tất số vòng lặp yêu cầu hoặc chạm `time_limit`, thuật toán chọn nút con của nút gốc có số lượt thăm `visits` lớn nhất (*Robust Child*). Nếu có nhiều nút cùng số lượt thăm, chọn nút có tỷ lệ thưởng trung bình `total / visits` cao hơn.

2.4.4 Mã giả (Pseudocode)

Trong mã giả, `stop()` kiểm tra thời hạn tìm kiếm và tín hiệu hủy; `rng` được khởi tạo theo cấu hình. Nút con mới chỉ được thêm vào cây sau khi rollout hoàn tất. Nếu vòng lặp bị dừng giữa chừng, khối `finally` hoàn tác mọi nước đi đã thực hiện và không cập nhật lan truyền ngược.

```
def MCTS_UCT(board, config, cancel):
    if board.is_terminal():
        return None

    root_moves = generate_candidates(board)
    root = Node(
        player_just_moved=-board.to_move,
        untried=list(root_moves),
    )

    for _ in range(config.iterations):
        if stop():
            break

        node, path, made = root, [root], 0
        interrupted = False
        try:
            # Selection
            while (not board.is_terminal()
                   and not node.untried
                   and node.children):
                if stop():
                    interrupted = True
                    break
                log_visits = log(max(1, node.visits))
                node = max(
                    node.children,
                    key=lambda child: (
                        child.total / child.visits
                        + config.exploration
                        * sqrt(log_visits / child.visits)
                    ),
                )
                board.play(node.move)
                made += 1
                path.append(node)
            if interrupted:
                break

            # Expansion
            expanded = None
            parent = node
            if not board.is_terminal() and node.untried:
                move = rng.choice(node.untried)
                board.play(move)
                made += 1
                expanded = Node(
                    player_just_moved=-board.to_move,
                    move=move,
                    untried=generate_candidates(board),
                )
                node = expanded
                path.append(node)

            # Simulation / Rollout
            rollout_steps = 0
            while (not board.is_terminal()
                   and rollout_steps < config.rollout_limit):
                if stop():
                    interrupted = True
                    break
                board.play(rng.choice(generate_candidates(board)))
                made += 1
                rollout_steps += 1
            if interrupted or stop():
                break
```

```
            # Attach only completed simulations, so UCT never divides by zero.
            if expanded is not None:
                parent.untried.remove(expanded.move)
                parent.children.append(expanded)

            # Backpropagation
            for visited in path:
                visited.visits += 1
                visited.total += rollout_value(
                    board, visited.player_just_moved
                )
        finally:
            for _ in range(made):
                board.undo()

    chosen = (
        max(
            root.children,
            key=lambda child: (child.visits, child.total / child.visits),
        )
        if root.children
        else None
    )
    return chosen.move if chosen else root_moves[0]
```

3 Thiết kế và Minh họa Giao diện

3.1 Mô hình kiến trúc 3 tầng (3-Layer Architecture)

Mã nguồn dự án được phân tách thành ba tầng chức năng nhằm tăng khả năng bảo trì, kiểm thử và mở rộng. Mỗi tầng đảm nhiệm một nhóm trách nhiệm riêng, hạn chế phụ thuộc trực tiếp giữa luật trò chơi, thuật toán tìm kiếm và giao diện.

1. **Tầng lõi trò chơi – Game Core** (`game/`)**:** Quản lý trạng thái bàn cờ thông qua `Board`, kiểm tra luật thắng–thua trong `rules.py` và sinh các nước đi ứng viên trong `candidates.py`. Tầng này chỉ phụ thuộc vào logic trò chơi, không phụ thuộc vào AI hay thư viện giao diện.

2. **Tầng bộ máy trí tuệ nhân tạo – AI Engine** (`ai/`)**:** Chứa ba tác tử tìm kiếm Backtracking, Alpha–Beta và MCTS; hàm lượng giá trong `evaluation.py`; cùng cấu hình `SearchConfig` và thống kê `SearchStats`. Mỗi tác tử nhận trạng thái bàn cờ và cấu hình tìm kiếm, rồi trả về nước đi kèm thống kê.

3. **Tầng giao diện và điều phối tiến trình – Presentation & Process Worker** (`ui/`)**:** `GameApp` trong `ui/app.py` xây dựng giao diện bằng Tkinter, nhận thao tác người dùng và hiển thị trạng thái ván đấu. `search_worker` trong `ui/worker.py` thực thi tác vụ tìm kiếm trong tiến trình riêng.

3.1.1 Cơ chế xử lý đa tiến trình không chặn giao diện (Non-blocking Multiprocessing)

Để giao diện không bị đóng băng khi AI tìm kiếm ở độ sâu lớn hoặc chạy nhiều vòng lặp MCTS, hệ thống thực hiện tác vụ AI bất đồng bộ trong một tiến trình riêng.
Tách tiến trình tính toán: Khi đến lượt AI, giao diện tạo tiến trình con bằng phương thức `spawn` của `multiprocessing` và chạy `search_worker` trong tiến trình đó. Thuật toán không chạy trên luồng sự kiện chính của Tkinter, nên cửa sổ vẫn phản hồi trong thời gian tìm kiếm. Việc tách tiến trình cũng đưa tính toán CPU ra khỏi luồng giao diện và GIL của tiến trình chính.
Trao đổi dữ liệu liên tiến trình (IPC): Hệ thống không truyền widget hay đối tượng giao diện Tkinter sang tiến trình con. Dữ liệu đầu vào chỉ gồm các giá trị có thể tuần tự hóa cần cho tìm kiếm, như kích thước bàn cờ `size`, mảng trạng thái `cells`, tên thuật toán và cấu hình `SearchConfig`. Tiến trình cha nhận kết quả qua `mp.Queue` và gửi yêu cầu hủy qua `mp.Event` (`cancel`).
Polling và đồng bộ phiên: Giao diện dùng `root.after(40, self._poll)` để kiểm tra hàng đợi định kỳ mỗi 40 ms mà không chặn vòng lặp sự kiện Tkinter. Mỗi yêu cầu tìm kiếm mang cặp định danh `(epoch, request_serial)`. Khi bắt đầu ván mới, hệ thống tăng `epoch` và phát tín hiệu `cancel` để yêu cầu hủy tác vụ cũ. Mỗi khi phát sinh yêu cầu tìm kiếm mới cho lượt AI, hệ thống tăng `request_serial`; khi người dùng bấm

Tạm dừng, giao diện bật tín hiệu `cancel` để yêu cầu hủy tác vụ tìm kiếm hiện tại. Giao diện chỉ nhận kết quả nếu cặp định danh khớp với yêu cầu đang hoạt động, nhờ đó loại bỏ kết quả trễ từ các tác vụ cũ.

3.2 Sơ đồ luồng xử lý

Luồng xử lý mô tả cách AI tìm nước đi bằng Negamax kết hợp cắt tỉa Alpha–Beta. Kết quả được chọn tốt nhất trong cây tìm kiếm đã duyệt, với độ sâu và tập nước đi ứng viên bị giới hạn theo cấu hình; do đó, thuật toán không khẳng định tối ưu tuyệt đối ngoài phạm vi tìm kiếm này.
Luồng tổng quát: Nhận trạng thái bàn cờ → lặp sâu dần → kiểm tra trạng thái kết thúc/độ sâu → sinh và duyệt nước đi → cập nhật cận và cắt tỉa → lưu kết quả của độ sâu hoàn tất → trả nước đi cho giao diện.

1. **Khởi tạo và tìm kiếm sâu dần (*Iterative Deepening*):** Khi đến lượt AI, bộ tìm kiếm nhận trạng thái bàn cờ, cấu hình thuật toán và giới hạn thời gian `time_limit`. Thuật toán tìm lần lượt ở độ sâu $1, 2, \ldots,$ `max_depth`. Trước mỗi lượt tìm kiếm và trong quá trình đệ quy, hệ thống kiểm tra thời gian cùng yêu cầu hủy. Nếu hết thời gian, ngoại lệ `SearchInterrupted` làm dừng lượt đang chạy; kết quả chưa hoàn tất của lượt đó bị loại bỏ, còn nước đi tốt nhất từ độ sâu hoàn tất gần nhất (`completed_depth`) được giữ lại.

2. **Kiểm tra trạng thái kết thúc hoặc giới hạn độ sâu:** Tại mỗi nút, thuật toán kiểm tra trạng thái thắng hoặc hòa trước khi mở rộng. Nếu đã có người thắng, Negamax trả về điểm có xét số ply để ưu tiên thắng sớm và trì hoãn thua. Theo quy ước của hàm tìm kiếm, trường hợp nút hiện tại là bên vừa thua trả về $-$`WIN_SCORE` $+$ `ply`; qua các mức đệ quy, dấu điểm được đổi để thể hiện góc nhìn của bên còn lại. Với `WIN_SCORE` $= 1{,}000{,}000$, trị tuyệt đối của điểm thắng/thua tương ứng là `WIN_SCORE` $-$ `ply`. Nếu ván hòa, hàm trả về 0. Nếu ván chưa kết thúc nhưng `depth == 0`, hàm gọi `evaluate(board, board.to_move)` để lượng giá thế cờ bằng heuristic cửa sổ 5 ô.

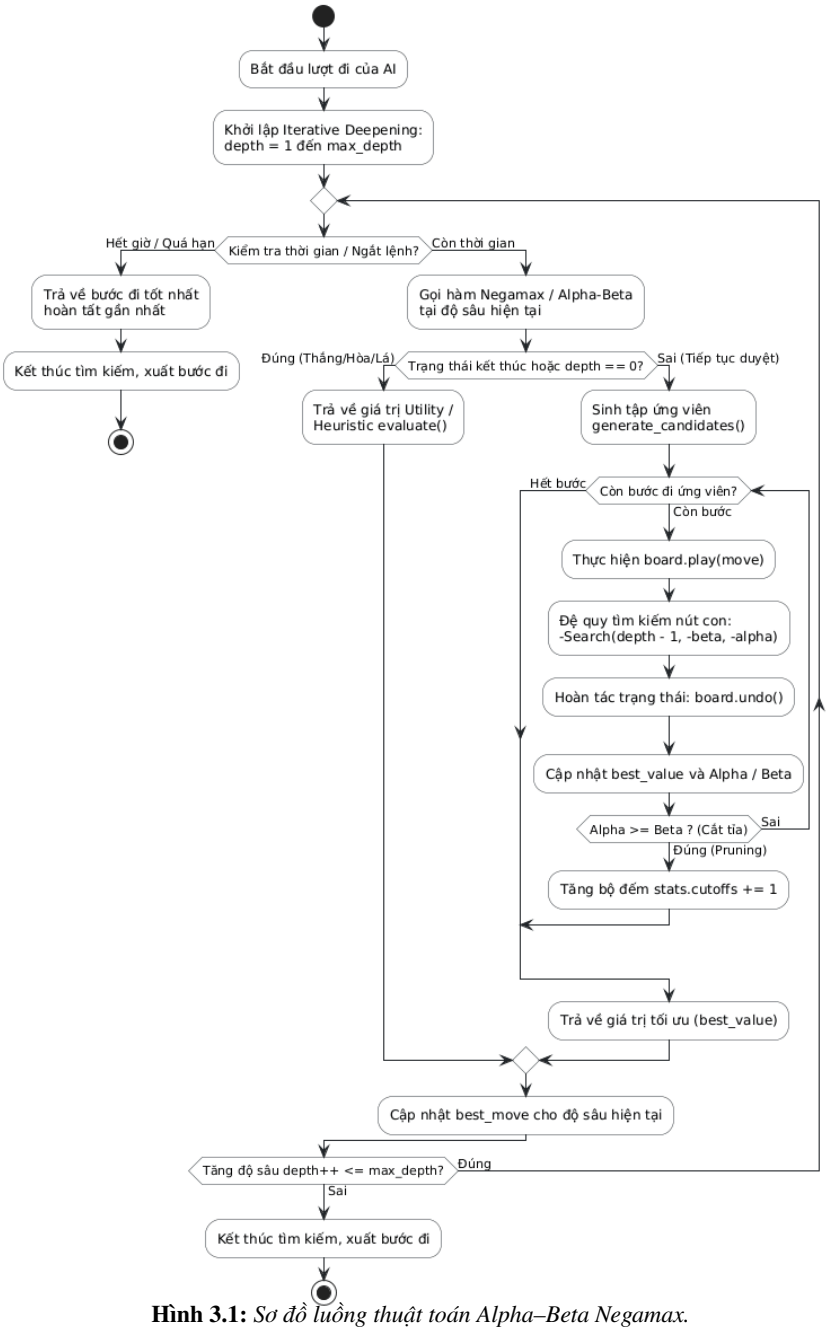
3. **Sinh nước đi ứng viên và duyệt nhánh:** Khi còn độ sâu tìm kiếm, thuật toán gọi `generate_candidates()` để giảm hệ số phân nhánh. Hàm áp dụng lần lượt các quy tắc sau:

   1. **Ưu tiên nước đi bắt buộc (*Forced Moves*):** Nếu bên đang tới lượt có nước thắng trực tiếp bằng cách tạo đủ năm quân liên tiếp, hàm chỉ trả về các nước thắng đó. Nếu không có nước thắng trực tiếp nhưng đối phương có thể thắng ngay ở lượt kế tiếp, hàm chỉ trả về các ô cần đánh để chặn các nước thắng ấy. Các nước thắng hoặc chặn bắt buộc không bị giới hạn bởi `max_candidates`.
   2. **Xử lý nước đi khai cuộc (*Opening*):** Nếu bàn cờ còn trống (`move_count == 0`), hàm trả về ngay ô chính giữa bàn cờ, có chỉ số tuyến tính `(size * size) // 2`.
   3. **Lọc theo khoảng cách Chebyshev ($R = 2$):** Với các ô còn lại, hàm chỉ giữ ô trống nằm cách không quá hai ô theo cả hàng và cột so với ít nhất một quân đã đánh. Cụ thể, với hai ô (r_1, c_1) và (r_2, c_2):
   $$d_\infty\big((r_1,c_1),(r_2,c_2)\big) = \max\big(|r_1 - r_2|, |c_1 - c_2|\big) \le 2.$$
   4. **Xếp hạng và giới hạn Top-K:** Các ứng viên được chấm điểm cục bộ bằng `priority`, sắp xếp theo thứ tự giảm dần và giữ tối đa `max_candidates` nước có điểm cao nhất. Mặc định, giới hạn này là $K = 12$ trên giao diện và $K = 8$ trong benchmark nhanh.

   Với từng ứng viên thu được, thuật toán đặt quân lên cùng đối tượng bàn cờ bằng `board.play(move)` rồi gọi đệ quy.

4. **Đệ quy Negamax và cắt tỉa Alpha–Beta:** Tại nút con, thuật toán gọi đệ quy với cửa sổ đã đổi dấu và hoán đổi vị trí $[-\beta, -\alpha]$. Sau khi nhận kết quả, `board.undo()` luôn được gọi trong khối `try...finally` để khôi phục trạng thái, kể cả khi tìm kiếm bị ngắt. Nút hiện tại cập nhật:
   $$\texttt{best} \leftarrow \max(\texttt{best}, \texttt{value}), \qquad \alpha \leftarrow \max(\alpha, \texttt{best}).$$
   Nếu $\alpha \ge \beta$, điều kiện *Beta Cutoff* được thỏa mãn; thuật toán tăng `stats.cutoffs` và dừng duyệt các ứng viên còn lại tại nút đó.

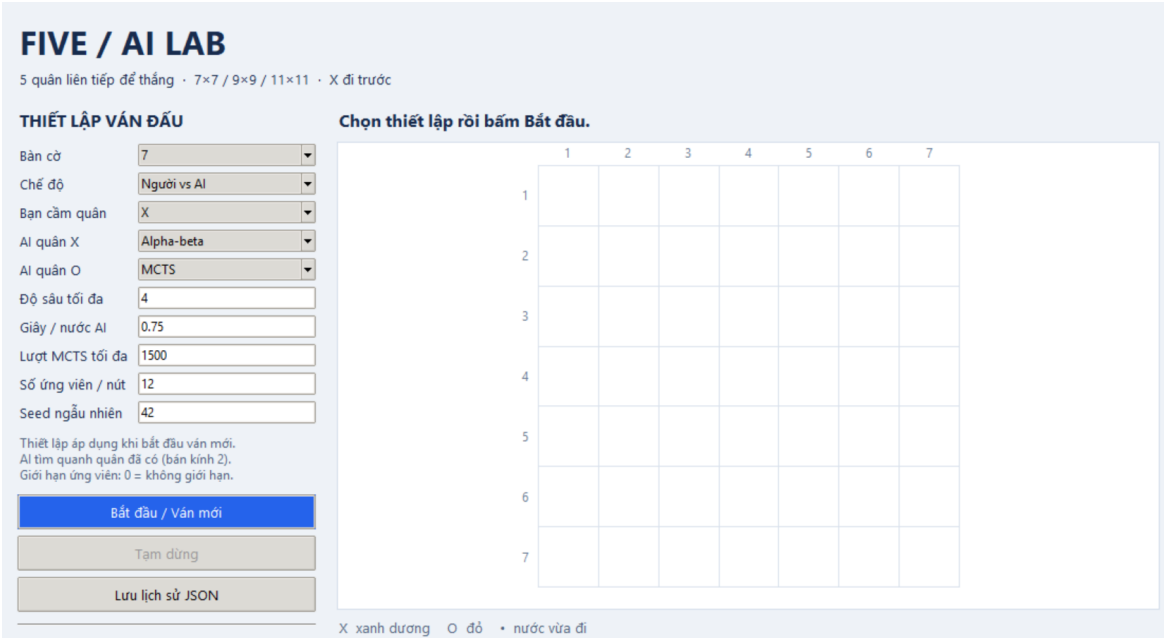
5. **Hoàn tất độ sâu và trả kết quả:** Chỉ khi toàn bộ lượt tìm kiếm ở một độ sâu hoàn tất, nước đi tốt nhất và điểm số của lượt đó mới được ghi nhận làm kết quả hiện hành. Nếu còn thời gian, chưa đạt `max_depth` và chưa có kết quả thắng/thua tuyệt đối, thuật toán tiếp tục ở độ sâu lớn hơn. Vòng lặp dừng khi hết thời gian, nhận yêu cầu hủy, đạt độ sâu tối đa hoặc khi:
   $$|\texttt{best_value}| \ge \texttt{WIN_SCORE} - N^2.$$
   Khi dừng, hệ thống trả về nước đi tốt nhất của độ sâu hoàn tất gần nhất cho giao diện.



Hình 3.1: *Sơ đồ luồng thuật toán Alpha–Beta Negamax.*

3.3 Giao diện người dùng

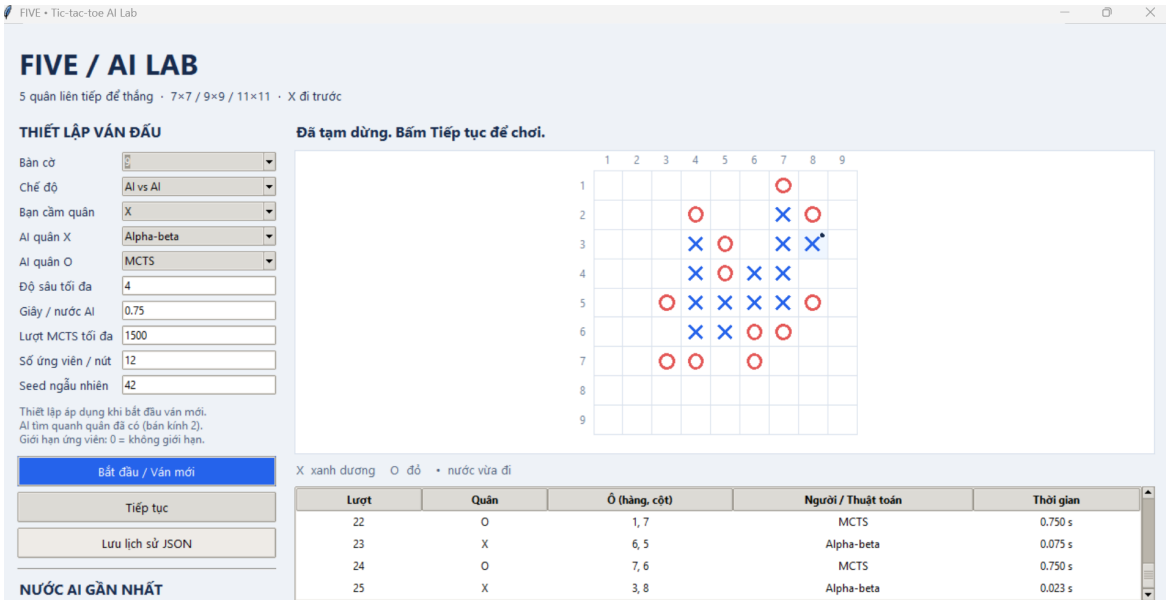
Hình 3.2 minh họa màn hình chính của ứng dụng FIVE–AI Lab. Khu vực bên trái chứa các thiết lập ván đấu và điều khiển, còn khu vực bên phải hiển thị bàn cờ cùng trạng thái lượt chơi. Người dùng có thể chọn kích thước bàn 7×7, 9×9 hoặc 11×11, chế độ chơi, màu quân và thuật toán AI.



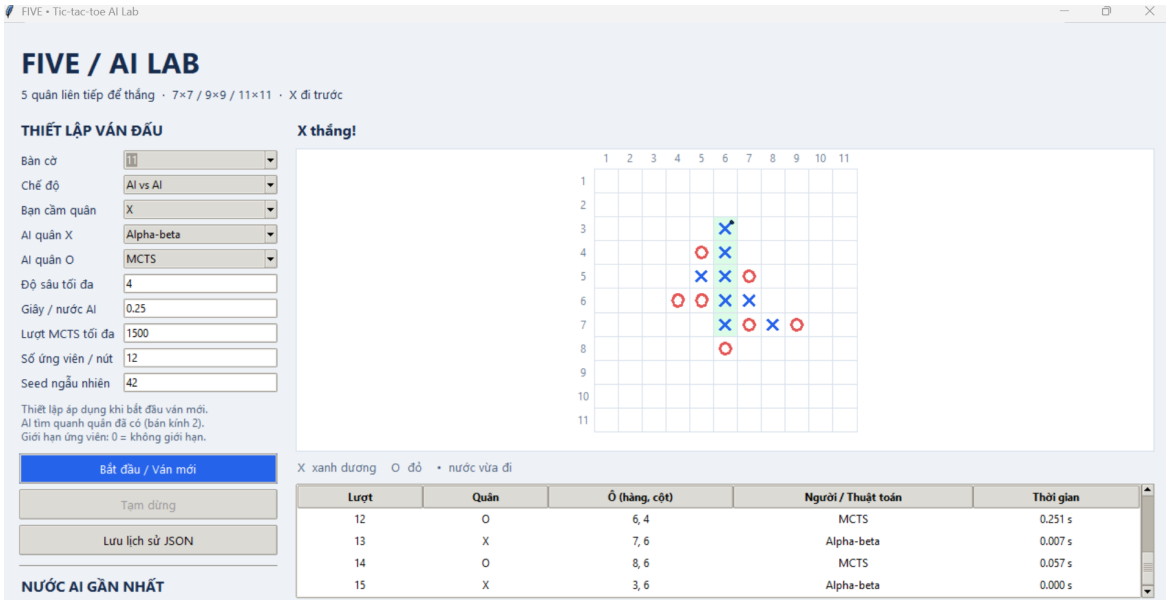
Hình 3.2: *Giao diện tổng quan ứng dụng FIVE–AI Lab với bảng thiết lập kích thước bàn cờ (7×7, 9×9, 11×11), chế độ chơi, tham số giới hạn tìm kiếm và tùy chọn các thuật toán AI.*



Hình 3.3: *Chế độ Người vs AI trên bàn cờ 7×7 (người cầm quân X xanh dương, tác tử MCTS cầm quân O đỏ). Giao diện làm nổi bật ô vừa đánh bằng nền xanh nhạt kèm chấm tròn và hiển thị nhật ký lượt đi ở bảng phía dưới.*



Hình 3.4: *Chế độ AI vs AI trên bàn cờ 9×9 giữa Alpha–Beta (quân X) và MCTS (quân O). Giao diện cho thấy ván đấu đang tạm dừng và nhật ký các lượt đi ở bảng phía dưới.*



Hình 3.5: *Màn hình kết thúc ván đấu trên bàn cờ mở rộng 11×11 (Alpha–Beta thắng MCTS). Hệ thống tự động phát hiện trạng thái kết thúc, khóa bàn cờ và tô sáng chuỗi năm quân X liên tiếp chiến thắng.*

Giao diện được tổ chức để người chơi thiết lập ván đấu ở bảng điều khiển bên trái và quan sát diễn biến trên bàn cờ bên phải. Trước khi bắt đầu, người dùng có thể chọn kích thước bàn, chế độ chơi, quân mình cầm và tác tử AI cho quân X hoặc O. Ba phương pháp AI có thể lựa chọn là Backtracking, Alpha–Beta và MCTS; nhờ đó, ứng dụng hỗ trợ cả chơi với máy lẫn cấu hình hai tác tử để thử nghiệm.

Nút *Bắt đầu / Ván mới* áp dụng cấu hình hiện tại và khởi tạo bàn cờ. Trong quá trình chơi, nút *Tạm dừng* điều khiển trạng thái ván; chức năng lưu lịch sử JSON hỗ trợ ghi lại dữ liệu để xem xét hoặc phục vụ thử nghiệm. Bàn cờ thể hiện các nước X và O, tọa độ hàng/cột và nước vừa đi, giúp người chơi theo dõi trạng thái ván trực quan.

4 Thực nghiệm và So sánh các phương pháp

Các thực nghiệm đánh giá hai khía cạnh: hiệu quả cắt tỉa khi Backtracking và Alpha–Beta tìm kiếm ở cùng độ sâu; và hiệu quả thi đấu của Backtracking, Alpha–Beta và MCTS dưới cùng ngân sách thời gian. Kết quả được trích xuất trực tiếp từ bộ thực nghiệm chuẩn trong `results/quick/` và đã được kiểm chứng tính toàn vẹn bằng `experiments/verify.py`.

4.1 Môi trường và thiết lập thử nghiệm

Thông tin môi trường được ghi trong metadata của lần chạy như sau:

Thành phần	Thông số được ghi nhận
Hệ điều hành	Windows 11, version 26H2, 64-bit (OS Build 26300)
Vi xử lý	Intel64 Family 6 Model 183 Stepping 1 (GenuineIntel)
Python	Python 3.11.7, GCC UCRT 13.2.0, 64-bit AMD64
Cách chạy	Tự động hóa bằng benchmark; mỗi lượt tìm kiếm chạy đơn luồng

Bảng 4.1: *Môi trường chạy thực nghiệm theo metadata.*

Các bàn cờ được thử nghiệm có kích thước 7×7, 9×9 và 11×11, điều kiện thắng là tạo được ít nhất năm quân liên tiếp. Hai seed được dùng là `11` và `28`. Các thiết lập thuật toán chính được tóm tắt trong Bảng 4.2.

Tham số	Giá trị	Ý nghĩa
Bán kính lân cận R	2	Lọc ứng viên theo khoảng cách Chebyshev quanh các quân đã đánh
Số ứng viên tối đa K	8	Giới hạn nhánh sau khi xếp hạng nước đi
Sắp xếp và nước bắt buộc	`True`	Ưu tiên nước thắng/chặn trực tiếp và sắp theo `priority`
Thí nghiệm độ sâu cố định	$d \in \{2,3\}$; không giới hạn thời gian	`iterative = False`; cùng thế cờ khai cuộc 10 ply
Thí nghiệm đối kháng	0.04 giây mỗi lượt	`max_depth = 20`; MCTS tối đa `1,000,000` vòng lặp, `rollout_limit = 60`
UCT của MCTS	$\sqrt{2}$	Hệ số cân bằng khai thác và khám phá

Bảng 4.2: *Các tham số cấu hình chính trong thực nghiệm.*

4.2 So sánh Backtracking và Alpha–Beta ở cùng độ sâu

Thí nghiệm thứ nhất đối chiếu hai thuật toán trên cùng trạng thái, seed và độ sâu. Mỗi trạng thái được tạo từ thế khai cuộc hợp lệ gồm 10 ply; giới hạn thời gian được tắt và tìm kiếm sâu dần không được sử dụng. Mục tiêu là đo số nút được duyệt, số nhánh bị cắt và thời gian thực thi khi giữ nguyên độ sâu.

Bảng 4.3 cho biết điểm Minimax, nước đi, số nút và tỷ lệ giảm nút. Bảng 4.4 trình bày thời gian đo được và tốc độ tương đối.

Bàn cờ	d	Seed	Score	Move	BT nodes	AB nodes	Cutoffs	Giảm nút (%)
7×7	2	11	-108	10	73	28	7	61.64
7×7	2	28	94	31	73	24	7	67.12
7×7	3	11	3982	10	585	102	13	82.56
7×7	3	28	4076	31	585	95	14	83.76
9×9	2	11	70	67	73	32	6	56.16
9×9	2	28	32	39	73	24	7	67.12
9×9	3	11	492	39	585	146	32	75.04
9×9	3	28	4070	39	585	95	14	83.76
11×11	2	11	320	49	59	31	5	47.46
11×11	2	28	126	70	73	28	7	61.64
11×11	3	11	8554	61	459	95	13	79.30
11×11	3	28	8124	72	585	116	19	80.17

Bảng 4.3: *So sánh số nút tìm kiếm ở độ sâu cố định.*

Bàn cờ	d	Seed	BT (ms)	AB (ms)	Tốc độ tăng
7×7	2	11	0.533	0.363	1.47×
7×7	2	28	0.481	0.352	1.37×
7×7	3	11	3.650	0.913	4.00×
7×7	3	28	3.703	0.965	3.84×
9×9	2	11	0.807	0.631	1.28×
9×9	2	28	0.772	0.600	1.29×
9×9	3	11	6.657	3.317	2.01×
9×9	3	28	6.123	1.652	3.71×
11×11	2	11	0.874	0.749	1.17×
11×11	2	28	1.297	0.997	1.30×
11×11	3	11	6.960	2.325	2.99×
11×11	3	28	10.372	3.802	2.73×

Bảng 4.4: *Thời gian thực thi và tốc độ tương đối của Alpha–Beta.*

Đối với các nút thông thường bị giới hạn tối đa $K = 8$ ứng viên, cây tìm kiếm đầy đủ có tối đa 73 nút ở độ sâu 2 và 585 nút ở độ sâu 3. Các nút xử lý nước thắng hoặc chặn bắt buộc có thể vượt giới hạn này.

Alpha–Beta kết hợp sắp xếp nước đi giảm số nút 47.46%–67.12% ở độ sâu 2 và 75.04%–83.76% ở độ sâu 3. Thời gian chạy nhanh hơn khoảng 1.17–1.47 lần ở độ sâu 2 và 2.01–4.00 lần ở độ sâu 3. Trên cả 12 cặp thử nghiệm, điểm Minimax và nước đi chọn được ghi nhận trùng khớp giữa hai phương pháp; kết quả này áp dụng trong cùng cây ứng viên và độ sâu đã thử, không khẳng định tối ưu ngoài phạm vi tìm kiếm.

Trong phép đo bổ sung ở độ sâu $d = 4$ với seed 11 (Bảng 4.10), Backtracking duyệt từ 2,325 đến 4,173 nút: 3,921 nút trên bàn 7×7, 4,173 nút trên bàn 9×9 và 2,325 nút trên bàn 11×11. Alpha–Beta duyệt tương ứng 325, 346 và 181 nút, giảm khoảng 91.7%–92.2% số nút so với Backtracking.

4.3 Đấu đối kháng dưới cùng ngân sách thời gian

Thí nghiệm thứ hai gồm 36 ván giữa ba thuật toán trên ba kích thước bàn cờ. Mỗi cặp thuật toán ở mỗi kích thước đấu bốn ván: hai seed và hai lượt đổi màu X/O; mỗi ván bắt đầu từ thế khai cuộc 4 ply. Tổng số trận là 3 cặp thuật toán $\times3$ kích thước $\times2$ seed $\times2$ lượt đổi bên $= 36$. Ngân sách mỗi lượt là 0.04 giây; các tác tử tìm kiếm cây có trần `max_depth = 20`, còn MCTS có trần `1,000,000` vòng lặp và `rollout_limit = 60`.

4.3.1 Thời gian phản hồi và lượng tìm kiếm

Bảng 4.5 tổng hợp thời gian trung bình, cực đại, tỷ lệ lượt chạm ngưỡng 40 ms và độ sâu hoàn tất trung bình. Bảng 4.6 tóm tắt lượng tìm kiếm trung bình trên mỗi lượt AI. Chỉ số số nút của MCTS là số nút cây; vì vậy không đồng nhất về ý nghĩa với số nút duyệt của hai phương pháp tìm kiếm đối kháng.

Bàn cờ	Thuật toán	Lượt AI	TB (ms)	Max (ms)	Chạm 40 ms	Độ sâu TB
7×7	Alpha–Beta	137	34.28	40.71	81.8%	7.28
7×7	Backtracking	148	35.99	40.21	89.2%	5.59
7×7	MCTS	131	40.02	40.20	100.0%	–
9×9	Alpha–Beta	70	28.73	40.08	67.1%	4.64
9×9	Backtracking	72	31.20	40.15	76.4%	3.43
9×9	MCTS	61	40.04	40.08	100.0%	–
11×11	Alpha–Beta	72	28.85	40.12	66.7%	4.46
11×11	Backtracking	77	32.55	40.18	77.9%	3.47
11×11	MCTS	49	40.06	40.12	100.0%	–

Bảng 4.5: *Thời gian và độ sâu tìm kiếm trung bình trong các ván đối kháng.*

Bàn cờ	Thuật toán	Nodes/lượt	Cutoffs/lượt	Nút cây MCTS	Iterations/lượt	rollout_moves
7×7	Alpha–Beta	3,778.2	1,047.2	–	–	–
7×7	Backtracking	7,213.9	0.0	–	–	–
7×7	MCTS	–	–	145.8	1,040.7	1,923.5
9×9	Alpha–Beta	1,570.4	424.7	–	–	–
9×9	Backtracking	3,114.6	0.0	–	–	–
9×9	MCTS	–	–	40.1	704.1	955.3
11×11	Alpha–Beta	1,151.1	272.6	–	–	–
11×11	Backtracking	2,302.5	0.0	–	–	–
11×11	MCTS	–	–	30.7	667.6	545.2

Bảng 4.6: *Khối lượng tìm kiếm trung bình trên mỗi lượt AI.*

MCTS sử dụng gần hết ngân sách thời gian ở cả ba kích thước; thời gian trung bình xấp xỉ 40 ms/lượt. Một số lượt vượt nhẹ mức 40 ms, phù hợp với việc đây là giới hạn thời gian mềm. Alpha–Beta đạt độ sâu hoàn tất trung bình cao hơn Backtracking khoảng 0.99–1.69 ply trong các cấu hình thử nghiệm. Kết quả này cùng với số nút duyệt thấp hơn cho thấy lợi ích của cắt tỉa trong ngân sách ngắn; mức cải thiện cụ thể thay đổi theo kích thước bàn cờ và thế cờ.

4.3.2 Kết quả thắng, hòa, thua

Bảng 4.7 tổng hợp thành tích của từng thuật toán theo kích thước bàn cờ. Tỷ lệ thắng được tính bằng W/n; tỷ lệ điểm được tính bằng $(W+0.5D)/n$, trong đó W, D lần lượt là số ván thắng và hòa, còn $n=8$ là số ván mỗi thuật toán ở mỗi kích thước.

Bàn cờ	Thuật toán	Số ván	Thắng	Hòa	Thua	Win rate	Score rate
7×7	Alpha–Beta	8	3	5	0	37.5%	68.75%
7×7	Backtracking	8	1	6	1	12.5%	50.00%
7×7	MCTS	8	0	5	3	0.0%	31.25%
9×9	Alpha–Beta	8	5	0	3	62.5%	62.50%
9×9	Backtracking	8	7	0	1	87.5%	87.50%
9×9	MCTS	8	0	0	8	0.0%	0.00%
11×11	Alpha–Beta	8	8	0	0	100.0%	100.00%
11×11	Backtracking	8	4	0	4	50.0%	50.00%
11×11	MCTS	8	0	0	8	0.0%	0.00%

Bảng 4.7: *Thành tích của từng thuật toán theo kích thước bàn cờ.*

Bảng 4.8 phân rã kết quả theo từng cặp đối đầu. Kết quả W–D–L được tính từ góc nhìn của thuật toán A trong cặp; cột X/O cho biết A cầm quân nào.

Bàn cờ	Cặp đấu (A–B)	A cầm X	A cầm O	Tổng W–D–L	Win rate A	Score rate A
7×7	Alpha–Beta–Backtracking	1–1–0	0–2–0	1–3–0	25.0%	62.5%
7×7	Alpha–Beta–MCTS	2–0–0	0–2–0	2–2–0	50.0%	75.0%
7×7	Backtracking–MCTS	1–1–0	0–2–0	1–3–0	25.0%	62.5%
9×9	Alpha–Beta–Backtracking	1–0–1	0–0–2	1–0–3	25.0%	25.0%
9×9	Alpha–Beta–MCTS	2–0–0	2–0–0	4–0–0	100.0%	100.0%
9×9	Backtracking–MCTS	2–0–0	2–0–0	4–0–0	100.0%	100.0%
11×11	Alpha–Beta–Backtracking	2–0–0	2–0–0	4–0–0	100.0%	100.0%
11×11	Alpha–Beta–MCTS	2–0–0	2–0–0	4–0–0	100.0%	100.0%
11×11	Backtracking–MCTS	2–0–0	2–0–0	4–0–0	100.0%	100.0%

Bảng 4.8: *Kết quả đối đầu trực tiếp theo cặp thuật toán và màu quân.*

Trong 36 ván, có 16 ván thắng bởi quân X, 12 ván thắng bởi quân O và 8 ván hòa; cả 8 ván hòa đều diễn ra trên bàn 7×7. Trên bàn này, Alpha–Beta bất bại với 3 thắng và 5 hòa. Trên bàn 9×9, Backtracking thắng 7/8 ván, trong khi Alpha–Beta thắng 5/8. Trên bàn 11×11, Alpha–Beta thắng cả 8 ván. Vì số ván và số seed còn hạn chế, các tỷ lệ này mô tả tập thực nghiệm chứ không đại diện cho xác suất thắng phổ quát.

4.3.3 Nhật ký 36 ván đấu

Bảng 4.9 ghi lại seed, thế khai cuộc 4 ply, cặp đấu và kết quả từng ván. BT là Backtracking, AB là Alpha–Beta; ký hiệu X/O chỉ màu quân của thuật toán thắng.

Bảng 4.9: *Nhật ký 36 ván đấu đối xứng theo seed.*

Bàn cờ	Seed	Khai cuộc (4 ply)	Cặp đấu	Ván 1 (bên 1 cầm X)	Ván 2 (đổi bên)
7×7	11	[24,12,38,11]	BT–AB	g001: hòa (49 ply)	g002: hòa (49 ply)
7×7	28	[24,22,33,47]	BT–AB	g003: hòa (49 ply)	g004: AB (X) thắng (23 ply)
7×7	11	[24,12,38,11]	BT–MCTS	g005: hòa (49 ply)	g006: hòa (49 ply)
7×7	28	[24,22,33,47]	BT–MCTS	g007: BT (X) thắng (11 ply)	g008: hòa (49 ply)
7×7	11	[24,12,38,11]	AB–MCTS	g009: AB (X) thắng (15 ply)	g010: hòa (49 ply)
7×7	28	[24,22,33,47]	AB–MCTS	g011: AB (X) thắng (23 ply)	g012: hòa (49 ply)
9×9	11	[40,24,59,26]	BT–AB	g013: BT (X) thắng (23 ply)	g014: AB (X) thắng (21 ply)
9×9	28	[40,38,49,57]	BT–AB	g015: BT (X) thắng (27 ply)	g016: BT (O) thắng (22 ply)
9×9	11	[40,24,59,26]	BT–MCTS	g017: BT (X) thắng (13 ply)	g018: BT (O) thắng (22 ply)
9×9	28	[40,38,49,57]	BT–MCTS	g019: BT (X) thắng (19 ply)	g020: BT (O) thắng (26 ply)
9×9	11	[40,24,59,26]	AB–MCTS	g021: AB (X) thắng (13 ply)	g022: AB (O) thắng (20 ply)
9×9	28	[40,38,49,57]	AB–MCTS	g023: AB (X) thắng (19 ply)	g024: AB (O) thắng (26 ply)
11×11	11	[60,40,69,45]	BT–AB	g025: AB (O) thắng (22 ply)	g026: AB (X) thắng (21 ply)
11×11	28	[60,58,71,51]	BT–AB	g027: AB (O) thắng (50 ply)	g028: AB (X) thắng (19 ply)
11×11	11	[60,40,69,45]	BT–MCTS	g029: BT (X) thắng (13 ply)	g030: BT (O) thắng (24 ply)
11×11	28	[60,58,71,51]	BT–MCTS	g031: BT (X) thắng (25 ply)	g032: BT (O) thắng (12 ply)
11×11	11	[60,40,69,45]	AB–MCTS	g033: AB (X) thắng (11 ply)	g034: AB (O) thắng (24 ply)
11×11	28	[60,58,71,51]	AB–MCTS	g035: AB (X) thắng (13 ply)	g036: AB (O) thắng (12 ply)

4.4 Mức tiêu thụ bộ nhớ

Backtracking và Alpha–Beta sử dụng một đối tượng bàn cờ, đặt và hoàn tác nước đi tại chỗ. Ngoài ngăn xếp đệ quy, mỗi mức tìm kiếm lưu danh sách ứng viên và một số biến cục bộ. Ở các nút thông thường, mỗi mức lưu tối đa K ứng viên nên bộ nhớ phụ trợ là $O(dK)$, tương đương $O(d)$ khi K là hằng số. Tại các nút xử lý nước thắng hoặc chặn bắt buộc, số ứng viên có thể vượt K; nếu mức i lưu K_i ứng viên thì cận tổng quát là $O(\sum_{i=1}^{d} K_i)$ (tối đa $O(dK_{\max})$, với $K_{\max}$ là số ứng viên lớn nhất ở một mức). Mức sử dụng này không tăng theo tổng số nút đã duyệt.

MCTS lưu các nút cây đã mở rộng cùng danh sách ứng viên chưa thử và các nút con. Với I vòng lặp, số nút cây có thể tăng theo I. Danh sách ứng viên ở nút thông thường bị giới hạn bởi K, nhưng các nút xử lý nước thắng hoặc chặn bắt buộc có thể vượt giới hạn này. Nếu $K_{\max}$ là số ứng viên lớn nhất được lưu tại một nút thực tế, bộ nhớ phụ thêm có cấp độ $O(IK_{\max})$; cận $O(IK)$ chỉ áp dụng khi mọi nút đều tuân theo giới hạn K. Bảng 4.10 tổng hợp phép đo `tracemalloc` trên thế cờ giữa ván 10 ply, seed 11, $K = 8$, sau khi đã khởi tạo bộ nhớ đệm hình học. Đây là lượng bộ nhớ cấp phát được Python theo dõi, không phải RSS tổng của tiến trình.

Bàn cờ	BT d=4 KB (nodes)	AB d=4 KB (nodes)	MCTS 1,000 vòng KB (nút cây)	MCTS/AB
7×7	6.30 (3,921)	5.99 (325)	328.88 (1,001)	54.9×
9×9	13.86 (4,173)	13.91 (346)	334.27 (1,001)	24.0×
11×11	15.18 (2,325)	14.94 (181)	327.90 (1,001)	21.9×

Bảng 4.10: *Bộ nhớ đỉnh được theo dõi bằng* `tracemalloc`.

Ở phép đo độ sâu 4, Backtracking và Alpha–Beta dùng lượng bộ nhớ cùng bậc, khoảng 6–15 KB, dù số nút được duyệt khác nhau. MCTS với 1,000 vòng lặp lưu 1,001 nút cây và dùng khoảng 328–334 KB, cao hơn Alpha–Beta khoảng 21.9–54.9 lần trong các phép đo này. Việc tăng số vòng lặp MCTS sẽ làm cây lớn hơn; mọi ngoại suy bộ nhớ từ các số liệu trên chỉ là ước lượng, không phải kết quả đo bổ sung.

4.5 Đánh giá tổng hợp và giới hạn

Trên bàn 7×7, 8/12 ván hòa và cả bốn ván có phân thắng bại đều do quân X thắng. Alpha–Beta bất bại trong tập trận này; MCTS cũng hòa ở các ván cầm X. Trên bàn 9×9, không có ván hòa; Backtracking thắng 7/8 ván, trong khi MCTS thua cả tám ván. Trên bàn 11×11, Alpha–Beta thắng cả tám ván. Đây là quan sát trên 36 ván với hai seed, không đủ để khái quát thành xác suất thắng cho mọi thế cờ.

Khi kích thước bàn tăng, không gian nước đi tiềm năng mở rộng và thời gian đánh giá mỗi nút cũng tăng. Trong giới hạn 40 ms, Alpha–Beta đạt độ sâu hoàn tất trung bình cao hơn Backtracking; tuy nhiên kết quả đối kháng trên bàn 9×9 cho thấy tìm sâu hơn không đồng nghĩa luôn thắng trong mọi cặp thử nghiệm. MCTS có tính anytime nhưng rollout bán ngẫu nhiên có thể bỏ qua các chuỗi đòn bắt buộc khi ngân sách thời gian ngắn. Các giải thích này nên được xem là diễn giải phù hợp với kết quả, không phải quan hệ nhân quả đã được kiểm chứng riêng.

Phương pháp	Ưu điểm	Hạn chế	Mức độ phù hợp
Backtracking	Cấu trúc đơn giản; dễ dùng làm baseline; bộ nhớ thấp nhờ đặt/gỡ quân tại chỗ.	Duyệt nhiều nút; độ sâu đạt được thấp hơn Alpha–Beta trong cùng ngân sách ở các cấu hình đã đo.	Thấp đến trung bình; phù hợp làm chuẩn đối chiếu hoặc tìm kiếm độ sâu nhỏ.
Alpha–Beta	Giữ giá trị Minimax trong cây tìm kiếm chung; cắt giảm số nút; có thể tìm sâu hơn trong cùng thời gian.	Hiệu quả phụ thuộc thứ tự nước đi; vẫn chịu ảnh hưởng của hàm heuristic và giới hạn tập ứng viên.	Cao trong các cấu hình thử nghiệm; đặc biệt hữu ích khi ngân sách thời gian hạn chế.
MCTS	Tìm kiếm anytime; tập trung mở rộng một phần cây thay vì duyệt đều mọi nhánh.	Cần lưu cây trong bộ nhớ; rollout có thể bỏ lỡ bẫy chiến thuật sâu khi thời gian ngắn.	Cần cải thiện policy/rollout hoặc tăng ngân sách trước khi kết luận trên bàn lớn.

Bảng 4.11: *Tổng hợp ưu điểm và hạn chế của ba phương pháp.*

Nhìn chung, Alpha–Beta giảm đáng kể số nút so với Backtracking ở cùng độ sâu và trong bộ trận này đạt kết quả tốt trên bàn 11×11. Tuy vậy, kích thước mẫu, số seed, giới hạn ứng viên $K = 8$ và ngân sách 40 ms là các giới hạn quan trọng. Vì vậy, kết luận chỉ áp dụng cho cấu hình đã mô tả; cần thêm seed, ván đấu và phép đo lặp để đánh giá độ ổn định và khả năng khái quát.

5 Tổng kết

5.1 Kết luận và hạn chế

Kết quả đạt được: Đề tài đã xây dựng hệ thống mô phỏng trò chơi cờ 5 quân trên các bàn cờ mở rộng kích thước 7×7, 9×9 và 11×11. Kiến trúc hệ thống tách biệt Game Core, AI Engine và giao diện Tkinter; tác vụ tìm kiếm được xử lý bất đồng bộ bằng tiến trình riêng để không làm gián đoạn tương tác của người dùng.

Về thuật toán, hệ thống sử dụng hàm lượng giá dựa trên các cửa sổ trượt gồm năm ô, trong đó điểm số được cập nhật tăng dần khi bàn cờ thay đổi. Do mỗi nước đi chỉ ảnh hưởng đến một số lượng cửa sổ hữu hạn với độ dài chuỗi thắng cố định, chi phí cập nhật là $O(1)$ theo kích thước bàn cờ. Bộ sinh nước đi kết hợp lọc lân cận Chebyshev với bán kính $R = 2$ và phát hiện các nước thắng hoặc chặn thắng bắt buộc nhằm giảm số ứng viên cần xét.

Trong cấu hình thực nghiệm của đề tài, Alpha–Beta giảm khoảng 75,04%–83,76% số nút duyệt ở độ sâu 3 so với Backtracking. Trong các ván đấu trên bàn 11×11, Alpha–Beta thắng cả 8/8 ván. Các kết quả này chỉ phản ánh phạm vi cấu hình và số ván đã thử nghiệm, không bảo đảm hiệu năng hoặc tỷ lệ thắng tương tự trên mọi thế cờ.

Hạn chế:

- Giới hạn vùng lân cận $R = 2$ và số ứng viên tối đa K là các phép xấp xỉ. Chúng giúp giảm hệ số phân nhánh nhưng có thể loại bỏ nước đi chiến lược ở xa trong một số thế cờ đặc biệt.
- Hệ thống chưa sử dụng Zobrist Hashing và Transposition Table, do đó các trạng thái tương đương đạt được qua những thứ tự nước đi khác nhau vẫn có thể bị tính toán lặp lại.
- MCTS hiện dùng rollout ngẫu nhiên có định hướng ở mức cơ bản nên có thể chưa nhận diện tốt các bẫy chiến thuật sâu, đặc biệt khi thời gian suy nghĩ bị giới hạn.

5.2 Hướng phát triển

- **Bảng băm trạng thái:** Bổ sung Zobrist Hashing và Transposition Table để lưu kết quả các trạng thái đã duyệt. Có thể kết hợp Killer Heuristic và History Heuristic để cải thiện thứ tự nước đi và tăng hiệu quả cắt tỉa Alpha-Beta.
- **Tìm kiếm không gian đe dọa:** Phát triển Threat-Space Search, tập trung vào các chuỗi tấn công bắt buộc như VCF (*Victory by Continuous Four*), nhằm phát hiện và xử lý các tình huống chiến thuật ở độ sâu lớn.
- **Nâng cấp MCTS:** Nghiên cứu RAVE (*Rapid Action Value Estimation*), PUCT kết hợp mạng nơ-ron theo hướng AlphaZero, hoặc song song hóa quá trình tìm kiếm bằng Root Parallelization hay Tree Parallelization.

5.3 Hướng dẫn cài đặt và khởi chạy

Yêu cầu môi trường: Python phiên bản ≥ 3.10. Dự án sử dụng thư viện chuẩn, bao gồm `tkinter`, không yêu cầu cài đặt thêm thư viện bên thứ ba qua `pip`.

Khởi chạy giao diện đồ họa: Mở terminal tại thư mục gốc dự án và chạy:

```
python main.py
```

Chạy kiểm thử tự động và kiểm tra giao diện:

```
python -m unittest discover -s tests -v
python -m tests.gui_smoke -v
```

Chạy benchmark và xác minh kết quả: Các lệnh sau lần lượt chạy benchmark nhanh với thời gian 0.04 giây mỗi nước và benchmark mở rộng với thời gian 0.5 giây mỗi nước:

```
# Benchmark nhanh (0.04 giay/nuoc)
python -m experiments.benchmark --output results/quick --time 0.04 --overwrite
python -m experiments.verify results/quick

# Benchmark mo rong (0.5 giay/nuoc, do sau 2-3-4)
python -m experiments.benchmark --output results/extended --time 0.5 --cap 12 --depths 2 3 4
python -m experiments.verify results/extended
```

6 TÀI LIỆU THAM KHẢO

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020, ch. 5, pp. 146–189.

[2] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[3] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. 17th European Conference on Machine Learning (ECML)*, Berlin, Germany: Springer, 2006, pp. 282–293.

[4] C. B. Browne et al., "A survey of Monte Carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012.

[5] L. V. Allis, "Searching for Solutions in Games and Artificial Intelligence," Ph.D. dissertation, Dept. Comput. Sci., Univ. Limburg, Maastricht, The Netherlands, 1994.

[6] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

[7] D. Silver et al., "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, Dec. 2018.

[8] Python Software Foundation, "Python 3 Standard Library Documentation: `tkinter` – Python interface to Tcl/Tk, and `multiprocessing` – Process-based parallelism," 2023.